شرح سطر بسطر ١١ — شاشة 27: إدخال الرمز السري و«نسيت الرمز؟»

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

حين يختار الطالب ملفًّا عليه رمز سري تظهر هذه الشاشة. هي قفل خفيف بين الإخوة وليست حماية أمنية قوية. تعتمد على `ProfileRepository.verifyPin` الذي يقرر: ناجح / خاطئ / مقفل (بعد 5 محاولات خاطئة لمدة 30 ثانية).

وإن نسي الطالب رمزه: «نسيت الرمز؟» تفتح ورقة تطلب رمز الطالب (`RZ-XXXX`)؛ إن صحّ يختار الطالب تعيين رمز جديد أو إلغاء الرمز، ثم يدخل مباشرة.

العدّ التنازلي المعروض (`_lockedSeconds`) محلي للعرض فقط؛ المرجع الحقيقي للقفل هو `PinLockout` داخل المستودع.

lib/features/profiles/pin_entry_screen.dart — شاشة إدخال الرمز

lib/features/profiles/pin_entry_screen.dart

منطق الرمز (التحقق، القفل المؤقت، «نسيت الرمز؟») لم يتغيّر. تغيّر الشكل حسب التصميم 05 و05ب: شريط علوي، التميمة في دائرة بيضاء وتصبح «مفكّرة» بعد الخطأ، شارة الاسم والصف، بطاقة خطأ حمراء، لوحة أرقام بحافة سفلية، وبطاقة «نسيت الرمز؟» العنبرية.

السطر 1

```
1   import 'dart:async';
```

ماذا يفعل؟ استيراد `dart:async` .

لماذا بهذه الطريقة؟ من أجل `Timer` الخاص بالعدّ التنازلي.

الأسطر 3–5

```
3   import 'package:flutter/material.dart';
4   import 'package:flutter_riverpod/flutter_riverpod.dart';
5   import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod وgo_router.

لماذا بهذه الطريقة؟ الشاشة تقرأ المزوّدات وتتنقّل.

الأسطر 7–21

```dart
import '../../app/routes.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/utils/pin_hasher.dart';
import '../../core/widgets/app_icon.dart';
import '../../core/widgets/icon_tile.dart';
import '../../core/widgets/mascot.dart';
import '../../core/widgets/pin_pad.dart';
import '../../core/widgets/screen_top_bar.dart';
import '../../core/widgets/tag_chip.dart';
import 'active_profile_provider.dart';
import 'forgot_pin_sheet.dart';
import 'profile_providers.dart';
import 'profile_repository.dart';
```

ماذا يفعل؟ استيراد المسارات والنصوص والألوان والأنماط وألوان الشخصية وأداة الرمز السري والشخصية ولوحة الرمز والشريط العلوي ومزوّدات الملفات وورقة «نسيت الرمز» والمستودع.

لماذا بهذه الطريقة؟ `profile_repository.dart` لنوع `PinCheck`.

الأسطر 23–27

```dart
/// إدخال الرمز السري لملف محمي — الشاشة 27.
class PinEntryScreen extends ConsumerStatefulWidget {
  const PinEntryScreen({super.key, required this.profileId});

  final String profileId;
```

ماذا يفعل؟ تعليق وصنف `PinEntryScreen` ومعاملها `profileId`.

لماذا بهذه الطريقة؟ تُفتح لملف بعينه.

الأسطر 29–31

```dart
  @override
  ConsumerState<PinEntryScreen> createState() => _PinEntryScreenState();
}
```

ماذا يفعل؟ `createState`.

لماذا بهذه الطريقة؟ للحالة المحلية.

السطر 33

```dart
class _PinEntryScreenState extends ConsumerState<PinEntryScreen> {
```

ماذا يفعل؟ صنف الحالة.

لماذا بهذه الطريقة؟ بحالة وبـ `ref`.

الأسطر 34–38

```
34    String _entry = '';
35    bool _wrong = false;
36    bool _busy = false;
37    int _lockedSeconds = 0;
38    Timer? _ticker;
```

ماذا يفعل؟ الحقول: الأرقام المُدخلة، وعلامة الخطأ، وعلامة الانشغال، والثواني المتبقية للقفل، والمؤقّت.

لماذا بهذه الطريقة؟ `_busy` يمنع الإدخال المزدوج أثناء التحقق.

السطر 40

```
40    bool get _isLocked => _lockedSeconds > 0;
```

ماذا يفعل؟ `_isLocked`: هل هناك ثوانٍ متبقية؟

لماذا بهذه الطريقة؟ خاصية مشتقة لا حالة مكرّرة.

الأسطر 42–46

```
42    @override
43    void dispose() {
44      _ticker?.cancel();
45      super.dispose();
46    }
```

ماذا يفعل؟ `dispose`: نلغي المؤقّت.

لماذا بهذه الطريقة؟ منع استدعاء `setState` بعد إغلاق الشاشة.

الأسطر 48–49

```
48    void _onDigit(String digit) {
49      if (_busy || _isLocked || _entry.length >= PinHasher.pinLength) return;
```

ماذا يفعل؟ `_onDigit`: نتجاهل الإدخال إن كان `_busy` أو مقفلًا أو اكتمل الطول.

لماذا بهذه الطريقة؟ أثناء القفل أو التحقق لا نقبل أرقامًا.

الأسطر 50–53

```
50      setState(() {
51        _wrong = false;
52        _entry += digit;
53      });
```

ماذا يفعل؟ نمسح علامة الخطأ ونضيف الرقم.

لماذا بهذه الطريقة؟ يبدأ الطالب محاولة جديدة.

الأسطر 54–55

```
54      if (_entry.length == PinHasher.pinLength) _verify();
55    }
```

ماذا يفعل؟ عند اكتمال أربعة أرقام نستدعي `_verify`.

لماذا بهذه الطريقة؟ لا زر «تأكيد»: التحقق تلقائي.

الأسطر 57–63

```
57    void _onDelete() {
58      if (_busy || _entry.isEmpty) return;
59      setState(() {
60        _wrong = false;
61        _entry = _entry.substring(0, _entry.length - 1);
62      });
63    }
```

ماذا يفعل؟ `_onDelete`: يحذف آخر رقم ويمسح علامة الخطأ.

لماذا بهذه الطريقة؟ إن كان `_busy` أو فارغًا لا يفعل شيئًا.

الأسطر 65–68

```
65    Future<void> _verify() async {
66      setState(() => _busy = true);
67      final repo = ref.read(profileRepositoryProvider);
68      final result = await repo.verifyPin(widget.profileId, _entry);
```

ماذا يفعل؟ `_verify`: نضع `_busy` ونستدعي `verifyPin` من المستودع.

لماذا بهذه الطريقة؟ المستودع يقرر النتيجة ويدير القفل.

السطر 69

```
69      if (!mounted) return;
```

ماذا يفعل؟ نخرج إن أُغلقت الشاشة أثناء الانتظار.

لماذا بهذه الطريقة؟ حماية `mounted`.

الأسطر 71–73

```
71      switch (result.check) {
72        case PinCheck.ok:
73          await _enter();
```

ماذا يفعل؟ حالة `ok`: ندخل.

لماذا بهذه الطريقة؟ `_enter` يجعل الملف نشطًا وينتقل للرئيسية.

الأسطر 74–79

```
74        case PinCheck.wrong:
75          setState(() {
76            _wrong = true;
77            _entry = '';
78            _busy = false;
79          });
```

ماذا يفعل؟ حالة `wrong`: نعلّم الخطأ ونفرّغ الإدخال ونرفع الانشغال.

لماذا بهذه الطريقة؟ النقاط تفرغ وتلوّن بالمرجاني فيرى الطالب أن عليه المحاولة ثانية.

الأسطر 80–87

```
80        case PinCheck.locked:
81          _startCountdown(result.lockedFor ?? const Duration(seconds: 30));
82          setState(() {
83            _wrong = false;
84            _entry = '';
85            _busy = false;
86          });
87      }
```

ماذا يفعل؟ حالة `locked`: نبدأ العدّ التنازلي بالمدة المتبقية ونفرّغ الإدخال.

لماذا بهذه الطريقة؟ 30 ثانية احتياطي إن لم تصل المدة.

السطر 88

```
88    }
```

ماذا يفعل؟ إغلاق `_verify`.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 90–93

```
90    Future<void> _enter() async {
91      await ref.read(activeProfileIdProvider.notifier).select(widget.profileId);
92      if (mounted) context.go(AppRoutes.home);
93    }
```

ماذا يفعل؟ `_enter`: نجعل الملف نشطًا ثم نفتح الرئيسية.

لماذا بهذه الطريقة؟ `go` فلا يرجع الطالب لشاشة الرمز بزر الرجوع.

الأسطر 95–97

```
95    void _startCountdown(Duration duration) {
96      _ticker?.cancel();
97      _lockedSeconds = (duration.inMilliseconds / 1000).ceil();
```

ماذا يفعل؟ `_startCountdown`: نلغي أي مؤقّت سابق ونحوّل المدة إلى ثوانٍ مقرّبة للأعلى.

لماذا بهذه الطريقة؟ مثلًا 29.4 ثانية تظهر 30.

الأسطر 98–102

```
98      _ticker = Timer.periodic(const Duration(seconds: 1), (timer) {
99        if (!mounted) return;
100       setState(() => _lockedSeconds = (_lockedSeconds - 1).clamp(0, 9999));
101       if (_lockedSeconds == 0) timer.cancel();
102     });
```

ماذا يفعل؟ مؤقّت دوري كل ثانية يخفض العدّاد حتى الصفر (مع `clamp`) ثم يتوقف.

لماذا بهذه الطريقة؟ عند الصفر تُفتح اللوحة تلقائيًا.

السطر 103

```
103   }
```

ماذا يفعل؟ إغلاق الدالة.

لماذا بهذه الطريقة؟ نهاية `_startCountdown` .

الأسطر 105–107

```
105   Future<void> _forgot() async {
106     final profile = ref.read(profileByIdProvider(widget.profileId)).value;
107     if (profile == null) return;
```

ماذا يفعل؟ `_forgot`: نقرأ الملف الحالي ونخرج إن لم يوجد.

لماذا بهذه الطريقة؟ لا ورقة بلا ملف.

الأسطر 108–112

```
108     final changed = await showForgotPinSheet(
109       context,
110       repository: ref.read(profileRepositoryProvider),
111       profile: profile,
112     );
```

ماذا يفعل؟ نفتح ورقة «نسيت الرمز؟» بالمستودع والملف.

لماذا بهذه الطريقة؟ تعيد `true` إن تغيّر الرمز.

الأسطر 113–114

```
113     if (changed && mounted) await _enter();
114   }
```

ماذا يفعل؟ إن تغيّر الرمز ندخل مباشرة.

لماذا بهذه الطريقة؟ أثبت الطالب ملكيته برمز الطالب.

الأسطر 116–118

```
116   @override
117   Widget build(BuildContext context) {
118     final profile = ref.watch(profileByIdProvider(widget.profileId)).value;
```

ماذا يفعل؟ `build` ونقرأ الملف بـ `watch`.

لماذا بهذه الطريقة؟ حين يصل الملف تُبنى الشاشة باسمه ولونه.

الأسطر 120–132

```
120     return Scaffold(
121       body: SafeArea(
122         child: CustomScrollView(
123           slivers: [
124             SliverToBoxAdapter(
125               child: ScreenTopBar(
126                 title: ArStrings.pinScreenTitle,
127                 showOffline: false,
128                 onBack: () => context.canPop()
129                     ? context.pop()
130                     : context.go(AppRoutes.profiles),
131               ),
132             ),
```

ماذا يفعل؟ `Scaffold` و`SafeArea` و`CustomScrollView`، وأوله الشريط العلوي «الرمز السري» بلا شارة «دون إنترنت»؛ الرجوع يعود للخلف أو لاختيار الملف.

لماذا بهذه الطريقة؟ الشريط في `SliverToBoxAdapter` حتى يكون جزءًا من التمرير.

الأسطر 133–142

```
133             SliverFillRemaining(
134               hasScrollBody: false,
135               child: Padding(
136                 padding: const EdgeInsetsDirectional.fromSTEB(20, 8, 20, 20),
137                 child: profile == null
138                     ? const SizedBox.shrink()
139                     : Column(
140                         crossAxisAlignment: CrossAxisAlignment.stretch,
141                         spacing: 12,
142                         children: [
```

ماذا يفعل؟ `SliverFillRemaining` (بلا جسم تمرير) يملأ باقي الشاشة؛ قبل وصول الملف لا شيء، ثم عمود بمسافة 12.

لماذا بهذه الطريقة؟ يملأ الشاشة فيدفع `Spacer` بطاقة «نسيت الرمز؟» إلى الأسفل، ويسمح بالتمرير إن صغرت الشاشة.

الأسطر 143–167

```
                // التميمة في دائرة بيضاء؛ «تفكّر» بعد رمز خاطئ.
                Center(
                  child: Container(
                    width: 116,
                    height: 116,
                    decoration: const BoxDecoration(
                      color: AppColors.card,
                      shape: BoxShape.circle,
                      boxShadow: [
                        BoxShadow(
                          color: AppColors.sky2,
                          offset: Offset(0, 4),
                        ),
                      ],
                    ),
                    alignment: Alignment.center,
                    child: Mascot(
                      size: 92,
                      color: profile.color,
                      mood: _wrong
                          ? MascotMood.thinking
                          : MascotMood.happy,
                    ),
                  ),
                ),
```

ماذا يفعل؟ دائرة بيضاء 116 بحافة سماوية، وفيها التميمة 92 بلون الطالب؛ مزاجها «تفكّر» بعد رمز خاطئ و«سعيدة» غير ذلك.

لماذا بهذه الطريقة؟ تغيّر وجه التميمة يقول «حاول مرة أخرى» بلطف دون لون أحمر مخيف للطفل.

الأسطر 168–176

```
                Center(
                  child: TagChip(
                    label: ArStrings.pinProfileChip(
                      profile.name,
                      ArStrings.gradeName(profile.grade),
                    ),
                    icon: 'user',
                  ),
                ),
```

ماذا يفعل؟ شارة «الاسم · الصف» بأيقونة شخص.

لماذا بهذه الطريقة؟ تؤكد للطالب أنه يفتح ملفه هو لا ملف أخيه.

الأسطر 177–190

```
                Text(
                  ArStrings.pinEnterTitle,
                  textAlign: TextAlign.center,
                  style: AppTextStyles.title26.copyWith(
                    color: _wrong ? AppColors.coral : AppColors.ink,
                  ),
                ),
                Text(
                  _wrong ? ArStrings.pinCalm : ArStrings.pinEnterSub,
                  textAlign: TextAlign.center,
                  style: AppTextStyles.body15.copyWith(
                    color: AppColors.soft,
                  ),
                ),
```

ماذا يفعل؟ العنوان «أدخل الرمز السري» (مرجاني بعد الخطأ)، وتحته جملة: «لا بأس، جرّب مرة أخرى» بعد الخطأ أو شرح قصير قبله.

لماذا بهذه الطريقة؟ النص يتغيّر مع الحالة فلا يحتاج الطفل إلى قراءة رسالة منفصلة.

الأسطر 191–197

```
                Center(
                  child: PinDots(
                    length: PinHasher.pinLength,
                    filled: _entry.length,
                    error: _wrong,
                  ),
                ),
```

ماذا يفعل؟ نقاط الرمز في المنتصف: عددها طول الرمز، والممتلئ منها بعدد ما أُدخل، وتحمرّ عند الخطأ.

لماذا بهذه الطريقة؟ النقاط تعرض التقدّم دون كشف الأرقام.

الأسطر 198–201

```
                _StatusCard(
                  wrong: _wrong,
                  lockedSeconds: _lockedSeconds,
                ),
```

ماذا يفعل؟ بطاقة الحالة: رسالة الخطأ أو العدّ التنازلي للقفل.

لماذا بهذه الطريقة؟ صنف منفصل يحجز مكانًا ثابتًا حين لا رسالة.

الأسطر 202–206

```
                PinPad(
                  onDigit: _onDigit,
                  onDelete: _onDelete,
                  enabled: !_isLocked && !_busy,
                ),
```

ماذا يفعل؟ لوحة الأرقام، معطّلة أثناء القفل أو التحقق.

لماذا بهذه الطريقة؟ منع ضغطات متكرّرة أثناء انتظار القاعدة.

الأسطر 207–218

```
                        const Spacer(),
                        _ForgotCard(onTap: _forgot),
                      ],
                    ),
                ),
              ),
            ],
          ),
        ),
      );
  }
}
```

ماذا يفعل؟ `Spacer` ثم بطاقة «نسيت الرمز؟»، ثم إغلاق الأقواس والصنف.

لماذا بهذه الطريقة؟ البطاقة في الأسفل بعيدًا عن الأرقام حتى لا تُلمس خطأً.

الأسطر 220–226

```
/// بطاقة الخطأ الحمراء (التصميم 05ب): أيقونة إعادة ورسالة الخطأ أو مهلة القفل.
/// تحجز ارتفاعًا صغيرًا ثابتًا حين لا رسالة حتى لا تقفز اللوحة.
class _StatusCard extends StatelessWidget {
  const _StatusCard({required this.wrong, required this.lockedSeconds});

  final bool wrong;
  final int lockedSeconds;
```

ماذا يفعل؟ تعليق وصنف `_StatusCard` بحقلَي الخطأ وثواني القفل.

لماذا بهذه الطريقة؟ رسالة واحدة تغطي الحالتين.

الأسطر 228–233

```
  @override
  Widget build(BuildContext context) {
    final text = lockedSeconds > 0
        ? ArStrings.pinLocked(lockedSeconds)
        : (wrong ? ArStrings.pinWrong : null);
    if (text == null) return const SizedBox(height: 8);
```

ماذا يفعل؟ النص: رسالة القفل إن كانت هناك ثوانٍ، وإلا رسالة الخطأ، وإلا لا شيء فنعيد فراغًا بارتفاع 8.

لماذا بهذه الطريقة؟ الفراغ الصغير الثابت يقلّل قفز اللوحة عند ظهور الرسالة.

الأسطر 235–264

```
    return DecoratedBox(
      decoration: BoxDecoration(
        color: AppColors.redTint,
        borderRadius: BorderRadius.circular(16),
      ),
      child: Padding(
        padding: const EdgeInsets.all(12),
        child: Row(
          spacing: 10,
          children: [
            Expanded(
              child: Text(
                text,
                style: AppTextStyles.label14.copyWith(color: AppColors.red),
              ),
            ),
            const IconTile(
              iconName: 'retry',
              iconColor: AppColors.red,
              color: AppColors.card,
              size: 36,
              iconSize: 18,
              radius: 12,
            ),
          ],
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ بطاقة حمراء فاتحة بزوايا 16: النص الأحمر ومربّع أيقونة «إعادة».

لماذا بهذه الطريقة؟ الأحمر الفاتح يلفت النظر دون أن يخيف، كما في التصميم 05ب.

الأسطر 266–270

```
/// بطاقة عنبرية: «نسيت الرمز؟» بأيقونة سؤال، تفتح ورقة الاستعادة.
class _ForgotCard extends StatelessWidget {
  const _ForgotCard({required this.onTap});

  final VoidCallback onTap;
```

ماذا يفعل؟ صنف `_ForgotCard` بدالة اللمس.

لماذا بهذه الطريقة؟ بطاقة مستقلة لأن التصميم يجعلها كتلة واضحة لا زرًّا نصيًّا.

الأسطر 272–303

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      child: Material(
        color: AppColors.amberTint,
        borderRadius: BorderRadius.circular(18),
        clipBehavior: Clip.antiAlias,
        child: InkWell(
          onTap: onTap,
          child: Padding(
            padding: const EdgeInsets.all(14),
            child: Row(
              spacing: 10,
              children: [
                Expanded(
                  child: Text(
                    ArStrings.pinForgot,
                    style: AppTextStyles.label14.copyWith(
                      color: AppColors.amberInk,
                    ),
                  ),
                ),
                const AppIcon('bulb', size: 22, color: AppColors.amberInk),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ `Material` عنبري بزوايا 18 و`InkWell`: نص «نسيت الرمز؟ اطلب من وليّ أمرك...» ومصباح.

لماذا بهذه الطريقة؟ اللون العنبري للمساعدة (لا للخطأ)، والمصباح يقول «هناك حل».

lib/features/profiles/forgot_pin_sheet.dart — «نسيت الرمز؟»

`lib/features/profiles/forgot_pin_sheet.dart`

الأسطر 1–2

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
```

ماذا يفعل؟ استيراد Material والخدمات.

لماذا بهذه الطريقة؟ `services` للمرشّحات.

الأسطر 4–11

```dart
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/utils/student_code.dart';
import '../../core/widgets/primary_button.dart';
import 'pin_setup_sheet.dart';
import 'profile_repository.dart';
```

ماذا يفعل؟ استيراد القاعدة (لنوع `Profile`) والنصوص والألوان والأنماط ورمز الطالب والزر وورقة اختيار الرمز والمستودع.

لماذا بهذه الطريقة؟ نعيد استعمال ورقة اختيار الرمز نفسها.

الأسطر 13–14

```dart
/// «نسيت الرمز؟»: يطلب رمز الطالب، فإن صحّ يتيح تعيين رمز جديد أو إلغاءه.
/// يعيد true إن تغيّر الرمز السري (فيدخل الطالب بعدها مباشرة).
```

ماذا يفعل؟ تعليق الدالة العامة.

لماذا بهذه الطريقة؟ تعيد `true` إن تغيّر الرمز السري.

الأسطر 15–19

```dart
Future<bool> showForgotPinSheet(
  BuildContext context, {
  required ProfileRepository repository,
  required Profile profile,
}) async {
```

ماذا يفعل؟ توقيع `showForgotPinSheet` بمعاملات مسمّاة: المستودع والملف.

لماذا بهذه الطريقة؟ تمرير المستودع بدل قراءته هنا يبقي الورقة بلا اعتماد على Riverpod فتُختبر بسهولة.

الأسطر 20–27

```dart
  final changed = await showModalBottomSheet<bool>(
    context: context,
    isScrollControlled: true,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
    ),
    builder: (_) => _ForgotPinSheet(repository: repository, profile: profile),
  );
```

ماذا يفعل؟ نفتح ورقة سفلية بخلفية `ground` وزوايا علوية 28.

لماذا بهذه الطريقة؟ `isScrollControlled` لترتفع فوق لوحة المفاتيح.

الأسطر 28–29

```
    return changed ?? false;
  }
```

ماذا يفعل؟ نعيد النتيجة أو `false` إن أُغلقت.

لماذا بهذه الطريقة؟ الإغلاق بلا اختيار = لم يتغيّر شيء.

الأسطر 31–39

```
class _ForgotPinSheet extends StatefulWidget {
  const _ForgotPinSheet({required this.repository, required this.profile});

  final ProfileRepository repository;
  final Profile profile;

  @override
  State<_ForgotPinSheet> createState() => _ForgotPinSheetState();
}
```

ماذا يفعل؟ ودجت `_ForgotPinSheet` بمستودع وملف و`createState`.

لماذا بهذه الطريقة؟ حالة محلية.

الأسطر 41–45

```
class _ForgotPinSheetState extends State<_ForgotPinSheet> {
  final _controller = TextEditingController();
  bool _verified = false;
  bool _wrongCode = false;
  bool _busy = false;
```

ماذا يفعل؟ الحالة: الـ controller وعلامات: تم التحقق، رمز خاطئ، انشغال.

لماذا بهذه الطريقة؟ تحدد أي واجهة تظهر.

الأسطر 47–51

```
  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
```

ماذا يفعل؟ `dispose`.

لماذا بهذه الطريقة؟ نتخلّص من الـ controller.

الأسطر 53–58

```
53    Future<void> _verify() async {
54      setState(() => _busy = true);
55      final ok = await widget.repository.matchesStudentCode(
56        widget.profile.id,
57        _controller.text,
58      );
```

ماذا يفعل؟ `_verify`: نضع الانشغال ونسأل المستودع هل الرمز يطابق.

لماذا بهذه الطريقة؟ `matchesStudentCode` تتحقق فقط ولا تغيّر شيئًا.

الأسطر 59–64

```
59      if (!mounted) return;
60      setState(() {
61        _busy = false;
62        _verified = ok;
63        _wrongCode = !ok;
64      });
```

ماذا يفعل؟ نحدّث الحالة: الانشغال انتهى، والتحقق حسب النتيجة، والخطأ عكسها.

لماذا بهذه الطريقة؟ إن صح الرمز تتبدّل الواجهة إلى الخيارين.

السطر 65

```
65    }
```

ماذا يفعل؟ إغلاق الدالة.

لماذا بهذه الطريقة؟ نهاية `_verify`.

الأسطر 67–69

```
67    Future<void> _setNewPin() async {
68      final pin = await showPinSetupSheet(context);
69      if (pin == null || !mounted) return;
```

ماذا يفعل؟ `_setNewPin`: نفتح ورقة اختيار رمز جديد ونخرج إن أُغلقت.

لماذا بهذه الطريقة؟ لا نغيّر شيئًا إن لم يختر الطالب.

الأسطر 70–76

```
70      await widget.repository.resetPinWithStudentCode(
71        widget.profile.id,
72        _controller.text,
73        newPin: pin,
74      );
75      if (mounted) Navigator.of(context).pop(true);
76    }
```

ماذا يفعل؟ نعيّن الرمز الجديد بالمستودع ثم نغلق الورقة بـ `true`.

لماذا بهذه الطريقة؟ المستودع يتحقق مجددًا من رمز الطالب قبل التغيير.

الأسطر 78–84

```
78    Future<void> _removePin() async {
79      await widget.repository.resetPinWithStudentCode(
80        widget.profile.id,
81        _controller.text,
82      );
83      if (mounted) Navigator.of(context).pop(true);
84    }
```

ماذا يفعل؟ `_removePin`: نلغي الرمز السري (بلا رمز جديد) ثم نغلق بـ `true`.

لماذا بهذه الطريقة؟ خيار «إلغاء الرمز السري».

الأسطر 86–90

```
86    @override
87    Widget build(BuildContext context) {
88      return SafeArea(
89        child: SizedBox(
90          width: double.infinity,
```

ماذا يفعل؟ `build` و`SafeArea` و`SizedBox` بعرض كامل.

لماذا بهذه الطريقة؟ العرض الكامل مطلوب كي لا تضيق الورقة.

الأسطر 91–97

```
91          child: Padding(
92            padding: EdgeInsetsDirectional.fromSTEB(
93              24,
94              24,
95              24,
96              16 + MediaQuery.viewInsetsOf(context).bottom,
97            ),
```

ماذا يفعل؟ حشوة 24/24/24/16 مع إضافة ارتفاع لوحة المفاتيح من `viewInsets`.

لماذا بهذه الطريقة؟ ترفع الورقة فوق لوحة المفاتيح.

الأسطر 98–102

```
98            child: Column(
99              mainAxisSize: MainAxisSize.min,
100             crossAxisAlignment: CrossAxisAlignment.stretch,
101             spacing: 14,
102             children: [
```

ماذا يفعل؟ `Column` بحجم أدنى ومسافة 14.

لماذا بهذه الطريقة؟ تأخذ الورقة ما تحتاجه فقط.

الأسطر 103–107

```
103              Text(
104                ArStrings.forgotTitle,
105                textAlign: TextAlign.center,
106                style: AppTextStyles.section18,
107              ),
```

ماذا يفعل؟ عنوان «استعادة الوصول».

لماذا بهذه الطريقة؟ نمط `section18` .

السطر 108

```
108              if (!_verified) ..._codeEntry() else ..._choices(),
```

ماذا يفعل؟ نعرض واجهة إدخال الرمز إن لم يُتحقَّق بعد، وإلا الخيارين.

لماذا بهذه الطريقة؟ `..._codeEntry()` و `..._choices()` تفرد القائمتين داخل الأبناء.

الأسطر 109–114

```
109            ],
110          ),
111        ),
112      ),
113    );
114  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `build` .

الأسطر 116–117

```
116  List<Widget> _codeEntry() {
117    const maxLength = 3 + StudentCode.randomLength;
```

ماذا يفعل؟ `_codeEntry` وطول رمز الطالب = البادئة (3) + الجزء العشوائي.

لماذا بهذه الطريقة؟ الطول من `StudentCode` فلو تغيّر لا ينكسر.

الأسطر 118–122

```
118    final borderColor = _wrongCode ? AppColors.coral : AppColors.line;
119    final border = OutlineInputBorder(
120      borderRadius: BorderRadius.circular(14),
121      borderSide: BorderSide(color: borderColor, width: 2),
122    );
```

ماذا يفعل؟ لون الحد (مرجاني للخطأ) وبناء الحد المشترك.

لماذا بهذه الطريقة؟ نفس نمط حقل رمز الصف.

الأسطر 124–125

```
    return [
      Text(ArStrings.forgotCodeLabel, style: AppTextStyles.body16Strong),
```

ماذا يفعل؟ قائمة تبدأ بتسمية «أدخل رمز الطالب».

لماذا بهذه الطريقة؟ نص فوق الحقل.

الأسطر 126–128

```
      SizedBox(
        height: 52,
        child: TextField(
```

ماذا يفعل؟ `SizedBox` ارتفاعه 52 و`TextField` .

لماذا بهذه الطريقة؟ ارتفاع Figma.

الأسطر 129–134

```
          controller: _controller,
          autofocus: true,
          textDirection: TextDirection.ltr,
          textCapitalization: TextCapitalization.characters,
          cursorColor: AppColors.teal,
          style: AppTextStyles.input17,
```

ماذا يفعل؟ الـ controller وتركيز تلقائي واتجاه LTR وأحرف كبيرة ولون مؤشر ونمط.

لماذا بهذه الطريقة؟ الرمز لاتيني فاتجاهه LTR حتى في واجهة RTL.

الأسطر 135–136

```
          // نعيد البناء مع كل حرف ليتفعّل زر «تحقّق» عند وجود نص.
          onChanged: (_) => setState(() => _wrongCode = false),
```

ماذا يفعل؟ `onChanged` : يمسح علامة الخطأ ويعيد البناء مع كل حرف.

لماذا بهذه الطريقة؟ بدونه لا يتفعّل زر «تحقّق» أبدًا لأن شرط التعطيل يُحسب عند البناء؛ وقد اكتشفنا هذا الخطأ باختبار.

الأسطر 137–143

```
          inputFormatters: [
            FilteringTextInputFormatter.allow(RegExp('[A-Za-z0-9-]')),
            LengthLimitingTextInputFormatter(maxLength),
            TextInputFormatter.withFunction(
              (old, value) => value.copyWith(text: value.text.toUpperCase()),
            ),
          ],
```

ماذا يفعل؟ مرشّحات: حروف وأرقام وشرطة فقط، وطول أقصى، وأحرف كبيرة.

لماذا بهذه الطريقة؟ تمنع الإدخال غير الصالح وتطابق شكل `RZ-XXXX` .

الأسطر 144–159

```
          decoration: InputDecoration(
            hintText: ArStrings.forgotCodeHint,
            hintStyle: AppTextStyles.input17.copyWith(
              color: AppColors.placeholder,
            ),
            filled: true,
            fillColor: AppColors.card,
            border: border,
            enabledBorder: border,
            focusedBorder: border.copyWith(
              borderSide: BorderSide(
                color: _wrongCode ? AppColors.coral : AppColors.teal,
                width: 2,
              ),
            ),
          ),
```

ماذا يفعل؟ زخرفة الحقل: نص إرشادي `RZ-XXXX` وخلفية بيضاء والحدود (تنقلب للمرجاني عند الخطأ).

لماذا بهذه الطريقة؟ مظهر متسق مع بقية الحقول.

الأسطر 160–161

```
        ),
      ),
```

ماذا يفعل؟ إغلاق الحقل.

لماذا بهذه الطريقة؟ نهاية `TextField`.

الأسطر 162–166

```
      if (_wrongCode)
        Text(
          ArStrings.forgotCodeWrong,
          style: AppTextStyles.label14.copyWith(color: AppColors.coral),
        ),
```

ماذا يفعل؟ إن كان الرمز خاطئًا نعرض «رمز الطالب غير صحيح.» بلون مرجاني.

لماذا بهذه الطريقة؟ رسالة واضحة.

الأسطر 167–170

```
      PrimaryButton(
        label: ArStrings.forgotVerify,
        onPressed: _busy || _controller.text.trim().isEmpty ? null : _verify,
      ),
```

ماذا يفعل؟ زر «تحقّق»: يُعطَّل أثناء الانشغال أو عند الحقل الفارغ.

لماذا بهذه الطريقة؟ `null` يعطّل الزر.

الأسطر 171–172

```
    ];
  }
```

ماذا يفعل؟ إغلاق القائمة والدالة.

لماذا بهذه الطريقة؟ نهاية `_codeEntry`.

الأسطر 174–180

```
  List<Widget> _choices() {
    return [
      Text(
        ArStrings.forgotChoose,
        textAlign: TextAlign.center,
        style: AppTextStyles.body15,
      ),
```

ماذا يفعل؟ `_choices`: نص «تم التحقق. ماذا تريد أن تفعل؟».

لماذا بهذه الطريقة؟ يظهر بعد نجاح التحقق.

السطر 181

```
      PrimaryButton(label: ArStrings.forgotSetNew, onPressed: _setNewPin),
```

ماذا يفعل؟ زر «تعيين رمز سري جديد».

لماذا بهذه الطريقة؟ يفتح ورقة اختيار الرمز.

الأسطر 182–187

```
      PrimaryButton(
        label: ArStrings.forgotRemove,
        onPressed: _removePin,
        backgroundColor: AppColors.mute,
        foregroundColor: AppColors.ink,
      ),
```

ماذا يفعل؟ زر «إلغاء الرمز السري» بخلفية `mute` ونص `ink`.

لماذا بهذه الطريقة؟ خيار ثانوي بمظهر أهدأ.

الأسطر 188–190

```
    ];
  }
}
```

ماذا يفعل؟ إغلاق القائمة والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.